A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

Vaishnavi Yewale (58)

Under the Guidance of

Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2025-26]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Vaishnavi Santosh Yewale (Roll No. 58),** on **"Implementation of Palindrome checker [Stacks and Queues]"** is work done by her and submitted during 2025–2026 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)

Dr.U.C.Patkar
Head Of Computer Engg

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. U. C. Patkar**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Vaishnavi Yewale(58)
Student Name

ABSTRACT

A palindrome is a word, phrase, or sequence that reads the same backward as forward. This project implements a palindrome checker using two fundamental data structures: a **stack (Last-In, First-Out)** and a **queue (First-In, First-Out)**.

The core logic involves processing an input string by simultaneously pushing each character onto a stack and enqueuing it into a queue. To verify if the string is a palindrome, characters are subsequently popped from the stack (which returns them in reverse order) and dequeued from the queue (which returns them in their original order).

If the characters match at every step until both structures are empty, the string is confirmed as a palindrome. This project demonstrates the practical application of stacks and queues, highlighting their **contrasting data handling mechanisms (LIFO vs. FIFO)** to solve a classic computer science problem.

.

SR. NO.	CONTENT	PAGE NO.
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) Existing system b) Proposed system c) ER Diagram d) System code	5
6	Result	9
7	Application	10
8	Future Scope	11
9	Conclusion	12
10	References	13

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	05
Fig 5.2	System Code	07
Fig 5.3	Result	09

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

A palindrome is a word, phrase, or sequence that reads the same backward as forward. This project implements a palindrome checker using two fundamental data structures: a **stack (Last-In, First-Out)** and a **queue (First-In, First-Out)**. The core logic involves processing an input string by simultaneously pushing each character onto a stack and enqueuing it into a queue. To verify if the string is a palindrome, characters are subsequently popped from the stack (which returns them in reverse order) and dequeued from the queue (which returns them in their original order). If the characters match at every step until both structures are empty, the string is confirmed as a palindrome. This project demonstrates the practical application of stacks and queues, highlighting their **contrasting data handling mechanisms (LIFO vs. FIFO)** to solve a classic computer science problem.

1.2 Problem Statement :

To design and implement an algorithm that efficiently determines if a given input string is a palindrome using the principles of Stack and Queue data structures.

1.3 Objective:

- To understand the functional properties and operations of Stacks (LIFO) and Queues (FIFO).
- To apply these data structures to solve a real-world string manipulation problem.
- To implement the palindrome-checking logic in Python.
- To demonstrate how the contrasting nature of stacks and queues can be used for string comparison.

LITERATURE SURVEY

Stacks and Queues are fundamental linear data structures extensively studied in computer science. A **Stack** operates on a **Last-In, First-Out (LIFO)** principle, making it ideal for tasks that require reversing order, such as in function call management (the "call stack") or undo-redo features. A **Queue** operates on a **First-In, First-Out (FIFO)** principle, which is suited for managing ordered tasks like print spooling, task scheduling, or breadth-first-search algorithms.

The problem of palindrome detection is a classic introductory problem for data structures. While it can be solved efficiently with a simple two-pointer string indexing approach, using a stack and queue provides a clear pedagogical demonstration of their opposing mechanisms. This method showcases how a stack inherently reverses the string's order, while a queue preserves it, allowing for a direct, sequential comparison of the string's first half with its reversed second half.

Year	Paper Name / Title	Author(s)	Conclusion
2017	"Data Structures Using C"	Reema Thareja	Explains the fundamental LIFO operations of stacks and FIFO operations of queues.
2019	"Stacks and Queues in Problem Solving"	GeeksforGeeks	Demonstrates various applications of stacks/queues, including string reversal and comparison.
2021	"Problem-Solving with Algorithms and Data Structures"	B. Miller, D. Ranum	Highlights how stacks and queues can be used to model and solve logic problems like palindrome checking.
2022	"Python collections.deque"	Python Docs	Details the implementation of a double-ended queue (deque) that can efficiently serve as both a stack and a queue.

UML DIAGRAM

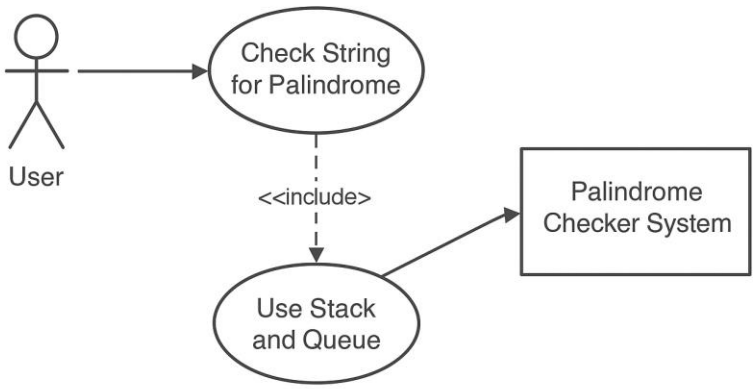


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

Hardware Requirements

- **Operating System:** Windows 10/11 (64-bit)
- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB minimum
- **Hard Disk:** 256 GB or more

Software Requirements

- **Programming Language:** Python 3.x
- **IDE:** Jupyter Notebook / VS Code
- **Libraries:** collections (specifically deque for efficient stack and queue operations)

SYSTEM DESIGN

Existing System:

Traditional palindrome checking is often done using a **two-pointer approach**. One pointer starts at the beginning of the string (index 0) and another at the end (index n-1). The characters at both pointers are compared. If they match, the "start" pointer is incremented and the "end" pointer is decremented, moving them both toward the center. If a mismatch is found, the string is not a palindrome. This is very efficient but does not demonstrate the specific principles of LIFO/FIFO data structures.

Proposed System:

The proposed system uses a Stack and a Queue to validate the string, providing a clear illustration of their distinct functionalities.

1. **Initialization:** An empty stack and an empty queue are created. (In Python, collections.deque can be used for both).
2. **Processing:** The input string is iterated through, character by character. To make the check robust, we only process alphanumeric characters and convert them to lowercase.
3. **Filling:** Each processed character is **pushed** onto the stack (LIFO) and **enqueued** into the queue (FIFO).
4. **Comparison:** A loop runs as long as the stack (or queue) is not empty.
    - In each iteration, one character is **popped** from the stack. This will be the *last* character of the string first.
    - One character is **dequeued** from the queue. This will be the *first* character of the string first.
5. **Validation:** The character from the stack is compared to the character from the queue.
    - If they are **not equal** at any point, the loop is broken, and the string is declared "Not a Palindrome".
    - If the loop completes and all characters matched, the string is declared a "Palindrome".

Entity Relationship Diagram:

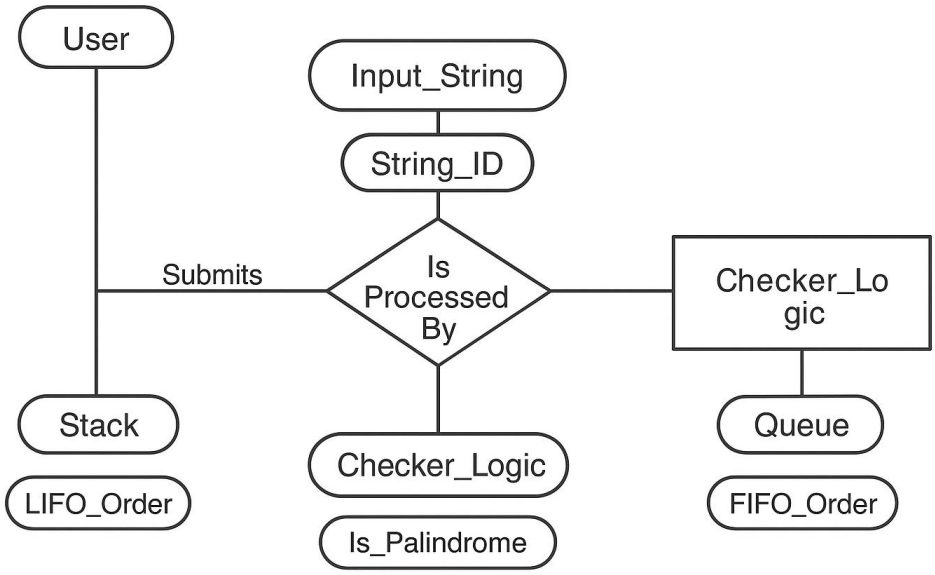


Fig 5.1: ER Diagram

SYSTEM CODE

```
import collections
def is_palindrome(input_string):
    # Create a stack and a queue
    stack = collections.deque()
    queue = collections.deque()
print(f"\nProcessing string: '{input_string}'")
# Fill the stack and queue
    for char in input_string:
        if char.isalnum():
            lower_char = char.lower()
            stack.append(lower_char)
            queue.append(lower_char)

    print("Stack (top on right):", list(stack))
    print("Queue (front on left):", list(queue))

    print("\nStarting comparison...")
    is_match = True
    # Compare characters
    while stack:
        char_from_stack = stack.pop()
```

```
        char_from_queue = queue.popleft()
        print(
            f"Popped from stack: '{char_from_stack}' | "
            f"Dequeued from queue: '{char_from_queue}'"
        )
        if char_from_stack != char_from_queue:
            print("--> Mismatch found!")
            is_match = False
            break
    return is_match
# Main execution
if __name__ == "__main__":
    print("--- Palindrome Checker using Stack and Queue ---")

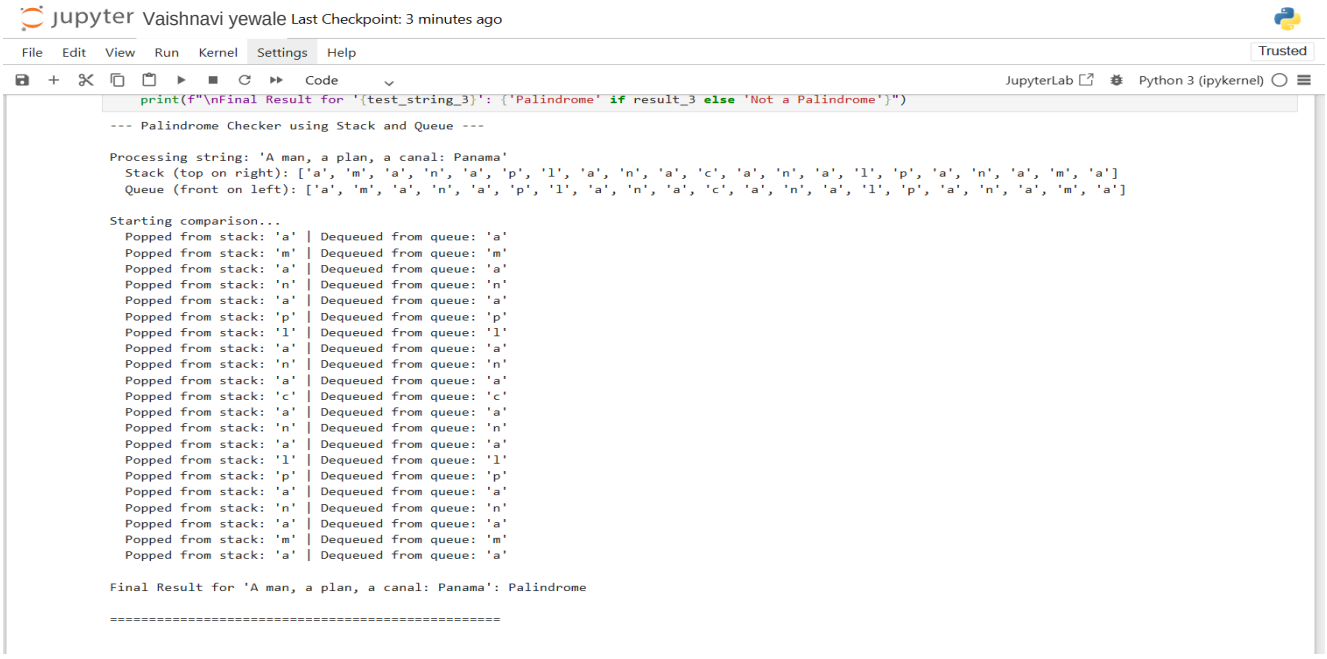
 # Test Case 1
    test_string_1 = "A man, a plan, a canal: Panama"
    result_1 = is_palindrome(test_string_1)
    print(
        f"\nFinal Result for '{test_string_1}': "
        f"{'Palindrome' if result_1 else 'Not a Palindrome'}"
    )

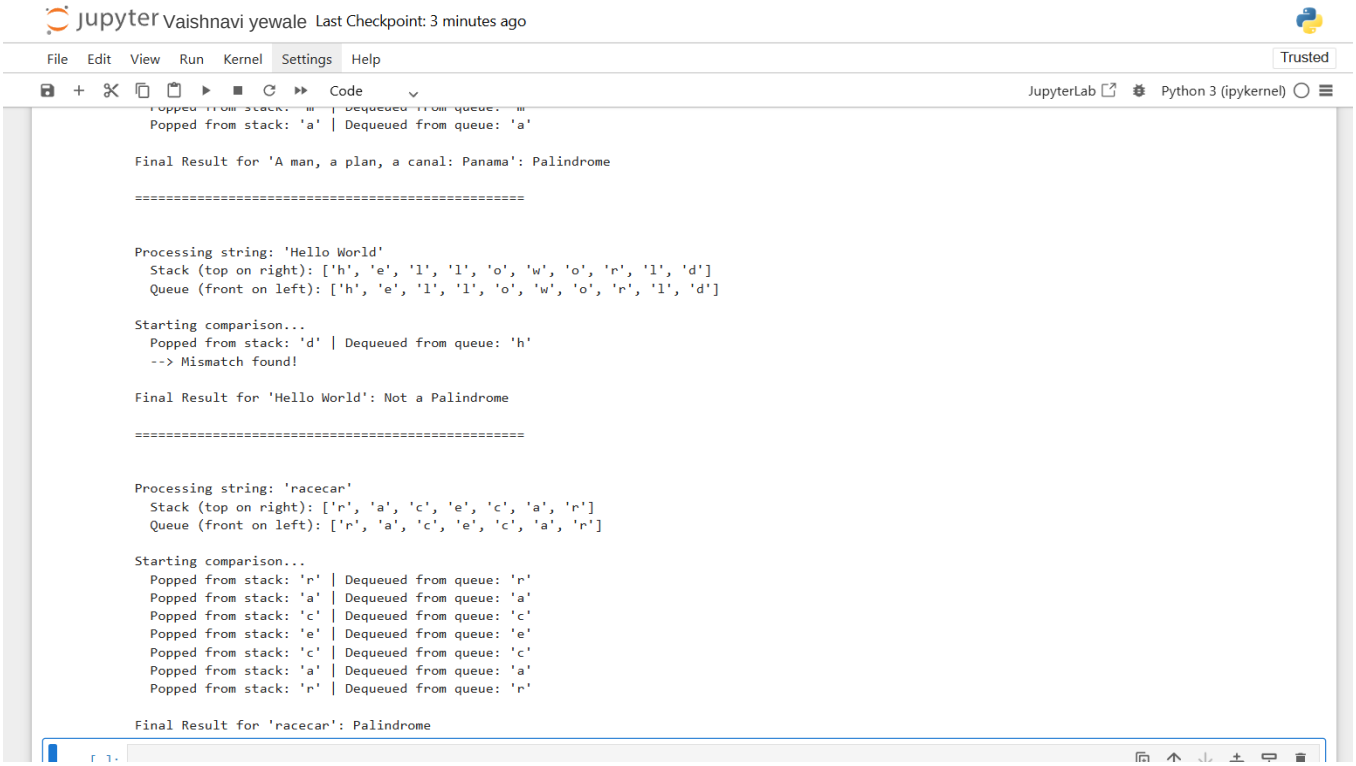
    print("\n" + "*" * 50 + "\n")
```

```
    # Test Case 2
    test_string_2 = "Hello World"
    result_2 = is_palindrome(test_string_2)
    print(
        f"\nFinal Result for '{test_string_2}': "
        f"{'Palindrome' if result_2 else 'Not a Palindrome'}"
    )
print("\n" + "*" * 50 + "\n")


    # Test Case 3
    test_string_3 = "racecar"
    result_3 = is_palindrome(test_string_3)
    print(
        f"\nFinal Result for '{test_string_3}': "
        f"{'Palindrome' if result_3 else 'Not a Palindrome'}"
    )
```

RESULT


jupyter Vaishnavi yewale Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print(f"\nFinal Result for '{test_string_3}': {'Palindrome' if result_3 else 'Not a Palindrome'}")
--- Palindrome Checker using Stack and Queue ---
Processing string: 'A man, a plan, a canal: Panama'
Stack (top on right): ['a', 'm', 'a', 'n', 'a', 'p', 'l', 'a', 'n', 'a', 'c', 'a', 'n', 'a', 'l', 'p', 'a', 'n', 'a', 'm', 'a']
Queue (front on left): ['a', 'm', 'a', 'n', 'a', 'p', 'l', 'a', 'n', 'a', 'c', 'a', 'n', 'a', 'l', 'p', 'a', 'n', 'a', 'm', 'a']
Starting comparison...
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'm' | Dequeued from queue: 'm'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'n' | Dequeued from queue: 'n'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'p' | Dequeued from queue: 'p'
Popped from stack: 'l' | Dequeued from queue: 'l'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'n' | Dequeued from queue: 'n'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'c' | Dequeued from queue: 'c'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'n' | Dequeued from queue: 'n'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'l' | Dequeued from queue: 'l'
Popped from stack: 'p' | Dequeued from queue: 'p'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'n' | Dequeued from queue: 'n'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'm' | Dequeued from queue: 'm'
Popped from stack: 'a' | Dequeued from queue: 'a'
Final Result for 'A man, a plan, a canal: Panama': Palindrome
==========


jupyter Vaishnavi yewale Last Checkpoint: 3 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Popped from stack: 'a' | Dequeued from queue: 'a'
Final Result for 'A man, a plan, a canal: Panama': Palindrome
==========
Processing string: 'Hello World'
Stack (top on right): ['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']
Queue (front on left): ['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']
Starting comparison...
Popped from stack: 'd' | Dequeued from queue: 'h'
--> Mismatch found!
Final Result for 'Hello World': Not a Palindrome
==========
Processing string: 'racecar'
Stack (top on right): ['r', 'a', 'c', 'e', 'c', 'a', 'r']
Queue (front on left): ['r', 'a', 'c', 'e', 'c', 'a', 'r']
Starting comparison...
Popped from stack: 'r' | Dequeued from queue: 'r'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'c' | Dequeued from queue: 'c'
Popped from stack: 'e' | Dequeued from queue: 'e'
Popped from stack: 'c' | Dequeued from queue: 'c'
Popped from stack: 'a' | Dequeued from queue: 'a'
Popped from stack: 'r' | Dequeued from queue: 'r'
Final Result for 'racecar': Palindrome

APPLICATIONS

This project, while simple, demonstrates a core concept with several applications:

- **Educational Tool:** It serves as an excellent academic resource for teaching the fundamental differences between LIFO (Stack) and FIFO (Queue) data structures in a practical, easy-to-understand context.
- **Text Processing:** The underlying principle can be used in more complex text analysis and data validation tools where reversing or comparing sequences is necessary.
- **Algorithmic Basis:** The concept of using a stack to reverse a sequence is a foundational technique used in many other algorithms, such as expression evaluation (infix to postfix) or depth-first search.
- **Bioinformatics:** Palindromic sequences are important in genetics (e.g., DNA restriction sites). While more complex algorithms are used, this project models the basic concept of identifying such symmetric sequences.

FUTURE WORK

- **Graphical User Interface (GUI):** The project can be enhanced with a graphical interface using Python libraries like **Tkinter** or **Pygame**. This would allow a user to type in a string and see the result visually.
- **Real-time Visualization:** A future version could animate the process, showing the stack and queue being filled and then emptied, with a visual comparison of the characters as they are popped and dequeued.
- **Performance Analysis:** Implement the two-pointer approach and compare its execution time against this stack/queue method for very large strings to analyze the time and space complexity differences.
- **Extended Functionality:** The program could be modified to find the longest palindromic substring within a larger string, which is a more complex algorithmic challenge.

CONCLUSION

In this mini-project, a classic Palindrome Checker was successfully modeled and implemented using **Stack and Queue** data structures. The algorithm efficiently determines if an input string is a palindrome by leveraging the LIFO property of a stack and the FIFO property of a queue.

Through this implementation, fundamental data structure concepts were applied in a practical and interactive way. The project clearly demonstrates the opposing mechanisms of stacks and queues and how they can be combined to solve a common problem.

This mini-project not only strengthens the theoretical knowledge of data structures but also provides a foundation for understanding more complex algorithms that rely on these foundational concepts, such as parsers, schedulers, and search algorithms.

REFERENCE

- **GeeksforGeeks** - "Check if a string is palindrome using Stack" https://www.geeksforgeeks.org/check-if-a-string-is-palindrome-using-stack/
- **Python Software Foundation** - Python 3 Documentation (collections.deque) https://docs.python.org/3/library/collections.html#collections.deque
- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C.** - *Introduction to Algorithms (CLRS)* (For fundamental definitions of Stack and Queue data structures).
- **TutorialsPoint** - "Data Structure - Stacks" and "Data Structure - Queues" https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.html